Architecture Specification: Tiddlywiki Tiddler Arrays

Generated from Markdown Specification

October 20, 2025

Abstract

This document outlines a detailed yet abstract architecture for representing documents as "Tiddlywiki Tiddler Arrays" and the software tools designed to convert between this format and others like Markdown, LaTeX, and PDF. The design is based on the idea of categories as different fields or axis inspired by the Concept Spaces (CSP) as described by Raubal [**?**] and leads the way to separate Tiddlers for each category like formulas, tables, pictures, bibliographic references, etc.

Contents

english**1 Tiddlywiki Tiddler Array: The Universal Document Representation** **3**
1.1 Tiddler Object Structure 3
1.2 Tiddlywiki Core Concepts in this Architecture 3

2 The Central Idea: From Publication to Reusable Knowledge Base **4**

3 General Program Structure & CSP Architecture **4**
3.1 Core Principles 4
3.2 Base Module (`CSP.ts`) 4

4 Software Development of Modules by LLM **5**

5 Conversion Workflows and Applications **5**
5.1 Honorable Mention: MathPix.com 5
5.2 Application Punchline: Automated Translation 5

6 Synergy and Final Punchline **5**

1 Tiddlywiki Tiddler Array: The Universal Document Representation

The core of this architecture is a universal, intermediate representation for documents, structured as a JSON array of "Tiddlers". Each tiddler is a granular piece of content (a paragraph, a formula, a section heading) enriched with metadata. This format decouples the document's structure and content from its final presentation.

1.1 Tiddler Object Structure

Each object in the array represents a tiddler and adheres to the following structure. All fields are strings.

Field	Description	Example
`title`	A unique identifier for the tiddler	`MyDoc2025_section.1`
`text`	The content of the tiddler in Markdown with KaTeX	'This is the first paragraph.'
`type`	The MIME type of the `text` field.	`text/markdown`
`kind`	The semantic type of the content. Used by processors.	`section`
`refnum`	A document-specific reference number.	`1.1`
`anchor`	A unique string for creating internal links or references.	`MyDoc2025_section.1`
`level`	For hierarchical elements like sections, indicates depth.	`1`
`originalLatex`	Stores the original, unprocessed LaTeX source.	`\section{Introduction}`
`tags`	TiddlyWiki-compatible tags for filtering.	`header bibkey:MyDoc2025`
custom fields	Modules can add any other fields as needed.	`page: "12"`

A document is simply a JSON file containing a single array of these tiddler objects: `[ {tiddler1}, {tiddler2}, ... ]`.

1.2 Tiddlywiki Core Concepts in this Architecture

A) Title as Unique Identifier The `title` field is the primary key for each tiddler. To ensure uniqueness across multiple documents and prevent collisions, a convention is used where the publication's `Bibkey` serves as a namespace. All tiddlers related to a specific document are prefixed with this key. For example: `Schu2013_section_1`, `Schu2013_formula_42`.

B) Transclusions and Template Transclusions Tiddlywiki's transclusion mechanism is fundamental to reassembling the document. Content from one tiddler is embedded within another.

- **Simple Transclusion**: A math expression stored in a tiddler titled `BibKey_FO0012` can be transcluded into a paragraph tiddler using the syntax {{`BibKey_FO012||FO`}}.
- **Template Transclusion**: This powerful feature combines a data tiddler with a template tiddler for rendering. For example, a tiddler representing a BibTeX-like entry can be combined with different templates to generate various outputs, such as an inline \`cite{bibkey}` command for LaTeX or a fully formatted entry for a bibliography list.

2 The Central Idea: From Publication to Reusable Knowledge Base

The workflow begins by finding a publication and creating a central, BibTeX-like tiddler for it. The subsequent goal is to find the publication's full text in a parsable format (PDF, LaTeX, etc.). This allows the system to ingest the entire content, breaking it down into a granular Tiddlywiki. This deep integration enables:

- **Content Reuse**: Text, and particularly math, can be easily reused across different contexts.
- **Math De-duplication**: Once all mathematical expressions are separated into their own tiddlers, identifying and de-duplicating identical expressions becomes a trivial task.

3 General Program Structure & CSP Architecture

The system is built on a modular pipeline where "Concept Space Processor" (CSP) modules operate sequentially on a data stream. This stream begins as a single text file and is progressively annotated and transformed by each module.

3.1 Core Principles

- **Modular & Extensible**: Each module is a self-contained class with a specific function. New features are added by creating new modules and adding them to the pipeline.
- **Pipeline Processing**: Modules are arranged in a specific order defined in a configuration file.
- **Stream-Based Communication**: Modules communicate by adding or modifying special metadata comments within the text stream.
- **Configuration Driven**: The entire pipeline is defined in a JSON configuration file.

3.2 Base Module (`CSP.ts`)

All processor modules inherit from a base `CSP` class, which provides a standard interface and common utilities.

```
// Abstract representation of the CSP class
export abstract class CSP {
    protected bibkey: string;
    protected flags: any;
    protected tiddlers: Map<string, Tiddler>;

    constructor(bibkey: string, tags: string, flags: any) {
        // ... initialization
    }

    // Processes the entire document stream
    abstract processFullText(text: string): string;

    // A secondary processing step for paragraph-level operations
    abstract processParagraph(paragraph: string): string;

    // Adds a new tiddler to the module's internal collection
    protected add(title: string, text: string, fields: object): void;

    // Appends the module's generated tiddlers to the main collection
    appendTiddlers(tiddlers: Map<string, Tiddler>): void;
}
```

Listing 1: Abstract representation of the CSP class

4 Software Development of Modules by LLM

A key part of the development workflow involves collaborating with Large Language Models (LLMs) to generate and debug CSP modules.

- **Code Exchange Protocol**: Source code for the entire project is exchanged within a single text block, where each file is demarcated by a header: `!!! path/to/filename.ts`.
- **LLM Constraints and Performance**: The rigid structure of the `CSP` base class serves as a strong constraint that guides the LLM. Experience has shown that various models can successfully adhere to this protocol.
- **Debugging Strategy**: When runtime errors do occur, the most effective debugging method is to use a simple, standalone test program that loads the problematic module directly.

5 Conversion Workflows and Applications

5.1 Honorable Mention: MathPix.com

A special acknowledgment goes to the MathPix OCR platform [**?**]. Their service is instrumental in converting PDF documents into a structured format, and their generous policy of allowing access to their CDN for all converted documents is a key enabler for this architecture.

5.2 Application Punchline: Automated Translation

The granular nature of the Tiddler Array lends itself to powerful applications. One such application is automated translation via services like DeepL.com [**?**]. The text content of each paragraph tiddler can be sent for translation, while the transclusion syntax is left intact.

6 Synergy and Final Punchline

The combination of the MathPix CDN, the discussed processing code, and Tiddlywiki's flexible interface creates a powerful synergy. It gives users the ability to generate tables that display a math expression's KaTeX rendering side-by-side with the cropped image of the OCR input from the original PDF. This provides an exceptional environment for quality control and the easy correction of OCR results.

References